音频背景知识

- 采样频率:每秒钟采样点的数量, 一般人声至少要5kHz , CD唱片是44.1kHz , 无损声音是48kHz , 超过48kHz 人耳无法识别而无效
- 采样精度(位宽): 用多少位的数据来表示一个音频数据点, 一般有8位的, 有16位的, 一般最大也就32位, 位数越多, 精度越高, 那么采集的数据也就越真实
- 声道数: 采集声音是多少个通道, 单声道听个响, 双声道可以判断左右耳声音距离, 四声道可以知道前后左右上下位置, 声道越多, 数据翻倍
- 比特率(码率): 音频每秒的数据量, 单位kbps(每秒多少千位, 不是字节) ,

  计算举例: 以48kHz的采样频率, 16位位宽, 双声道为例

  那么就是1s 1个声道有48k的采样点, 每个采样点是2字节, 双声道

  比特率= 48*2*2*8 = 1536kbps

引入SDL

音频解码, 需要ffmpeg , 音频播放需要sdl

添加依赖路径以及引入库

将ffmpeg和SDL2拷贝到工程目录下，在工程的pro文件中加入：

```
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include\
            $$PWD/SDL2-2.0.10/include

LIBS += $$PWD/ffmpeg-4.2.2/lib/avcodec.lib\
        $$PWD/ffmpeg-4.2.2/lib/avdevice.lib\
        $$PWD/ffmpeg-4.2.2/lib/avfilter.lib\
        $$PWD/ffmpeg-4.2.2/lib/avformat.lib\
        $$PWD/ffmpeg-4.2.2/lib/avutil.lib\
        $$PWD/ffmpeg-4.2.2/lib/postproc.lib\
        $$PWD/ffmpeg-4.2.2/lib/swresample.lib\
        $$PWD/ffmpeg-4.2.2/lib/swscale.lib\
        $$PWD/SDL2-2.0.10/lib/x86/SDL2.lib
```

包含头文件

```
extern "C"
{
    #include "libavcodec/avcodec.h"
    #include "libavformat/avformat.h"
    #include "libavutil/pixfmt.h"
    #include "libswscale/swscale.h"
    #include "libavformat/avformat.h"
    #include "libavdevice/avdevice.h"
    #include <SDL.h>
}
```

注意事项 使用sdl 会提示main函数没有定义的错误

原因是 SDL_main.h中有如下一段话：

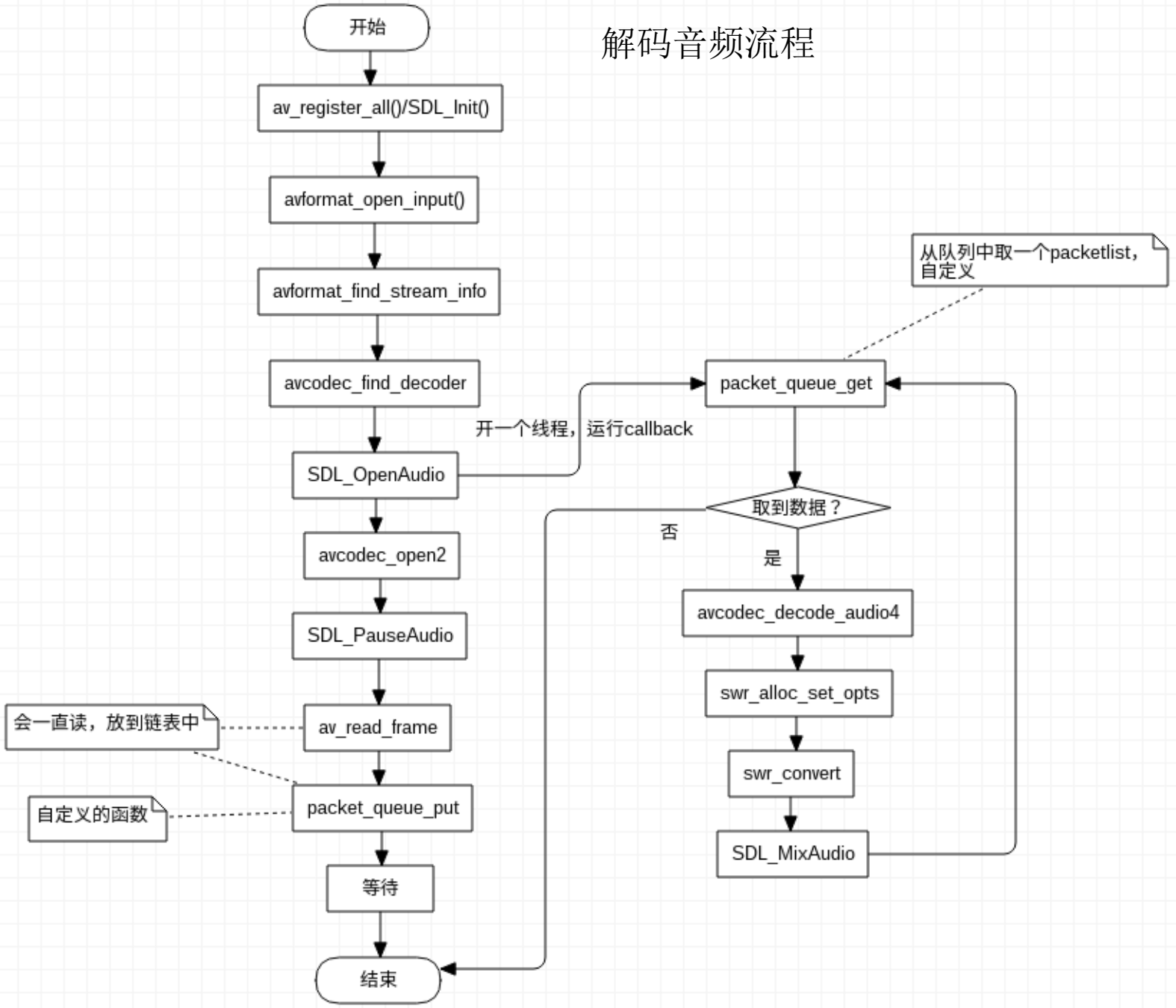
```
#if defined(SDL_MAIN_NEEDED) || defined(SDL_MAIN_AVAILABLE)
#define main    SDL_main
#endif
```

解决办法

```
#undef main
int main(int argc, char *argv[])
{

}
```

解码音频流程



//0.申请变量
//1.ffmpeg 初始化
//2.SDL初始化
//3.打开文件
//3.1获取文件流信息
//4.读取音频流
//5.找到对应的解码器
//6.设置音频信息,用来打开音频设备。
//7.打开音频设备。
//8.设置参数，供解码时候用
//9.打开解码器
//10.初始化队列
//11.SDL播放音频
//12.循环读取音频帧,放到音频同步队列中
//13.回调函数中将从队列中取数据,解码后填充到播放缓冲区.

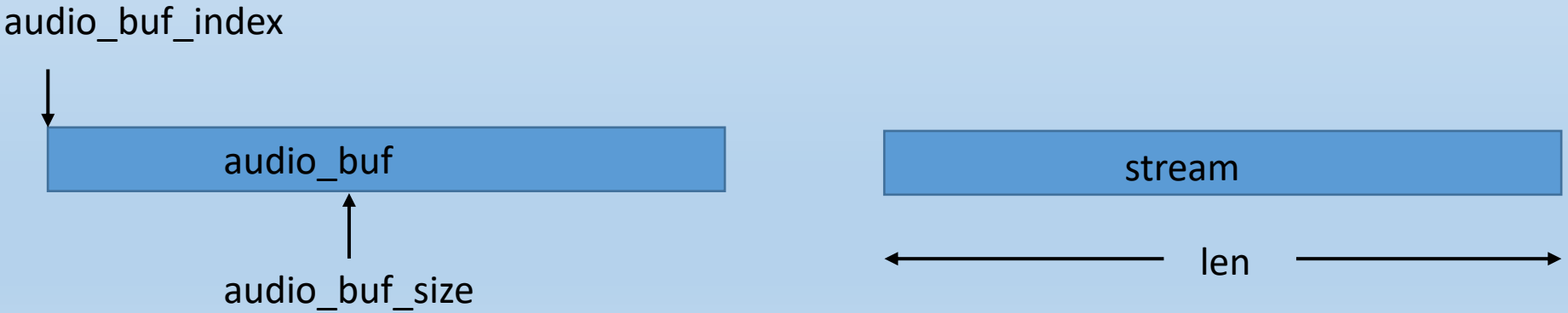
音频解码难点 – 回调函数实现

void audio_callback(void *userdata, uint8 *stream, int len)

参数 userdata 是回调自定义参数, 此时是解码器 stream是要播放的缓冲区, 需要用户为其赋值 len是你要填充的长度

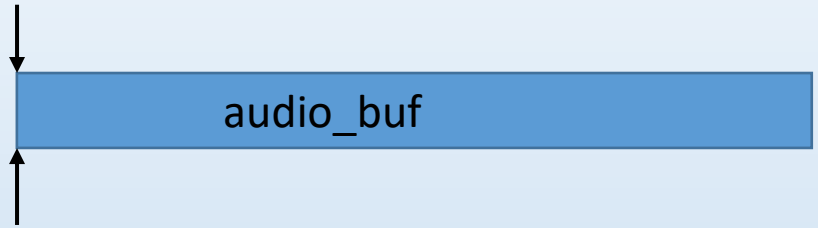
len的长度受位宽、声道数影响 , 一般采样点是1024个, 那么 16位, 双声道的音频, len的大小是4096 或8192等2的整数次幂

定义缓存空间
audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2]; 大小是最大音频空间的1.5倍, 目的不溢出
audio_buf_size 表示 缓冲空间中的数据长度
audio_buf_index 表示 从什么位置开始拷贝



audio_buf_index >= audio_buf_size 说明已经没有数据可以拷贝，需要解码得到一些音频数据

audio_buf_index

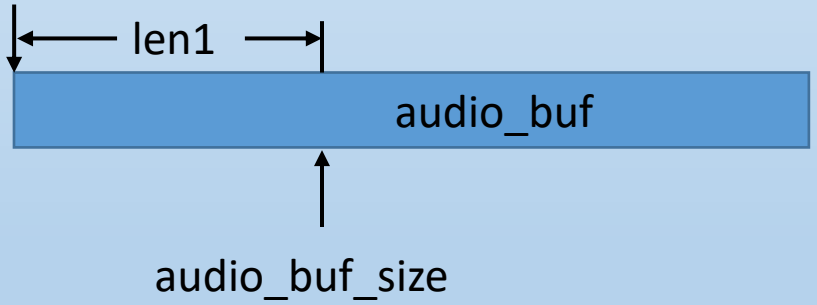


audio_buf_index >= audio_buf_size 要去解码数据的标志

audio_buf_size

解码一些数据，并且每次解码后 audio_buf_index都会归0，解码后的数据会复制到audio_buf中从0开始到audio_buf_size结束

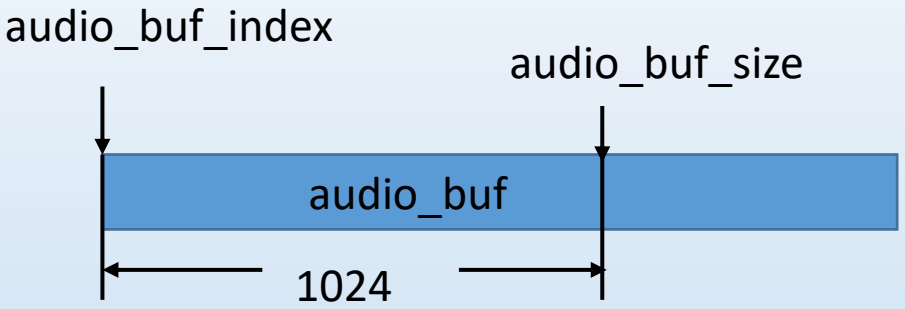
audio_buf_index



audio_buf_size

要复制的长度 len1 = audio_buf_size – audio_buf_index

调用解码, 如果解码返回-1 , 即解码失败, 默认是静音

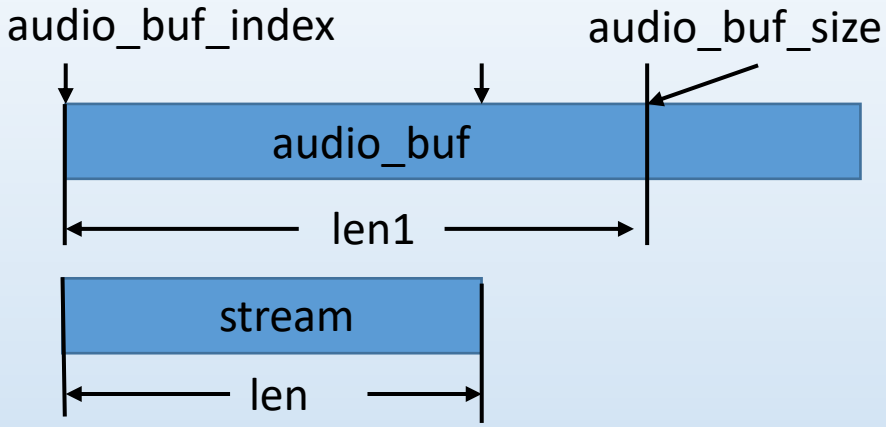


会让audio_buf_size = 1024 ,
让其缓冲区audio_buf内容都是0 , 播放效果为静音

代码为

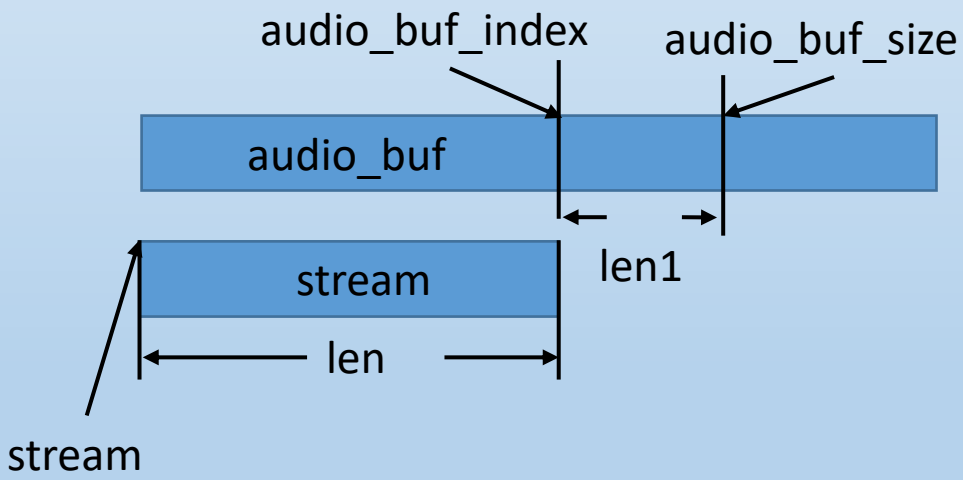
```
audio_data_size = audio_decode_frame( is,audio_buf,sizeof(audio_buf));
if (audio_data_size < 0)
{
 /* silence */ audio_buf_size = 1024; /* 清零，静音 */
            memset(audio_buf, 0, audio_buf_size);
} else {
            audio_buf_size = audio_data_size;
}
audio_buf_index = 0;
```

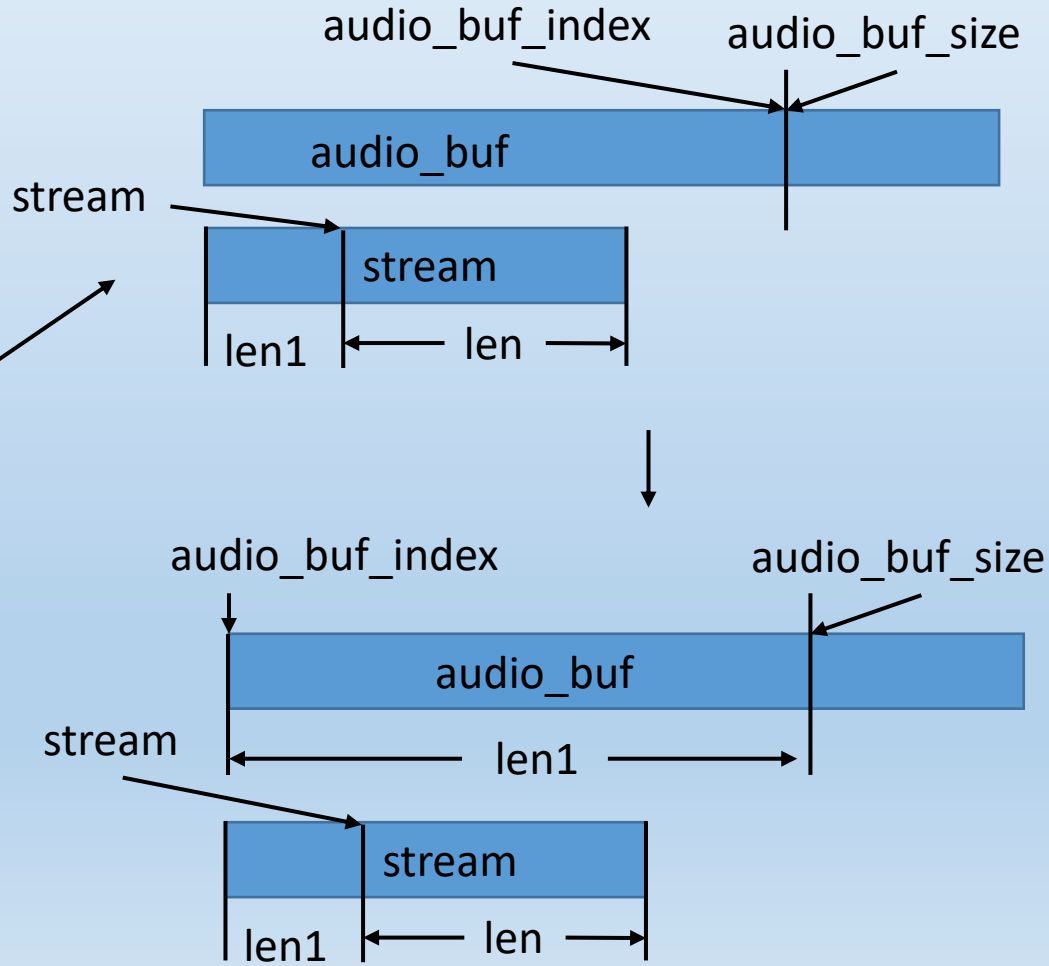
len1 > len 时 表示此时stream复制满, 还剩余一些, 下一次回调拷贝.



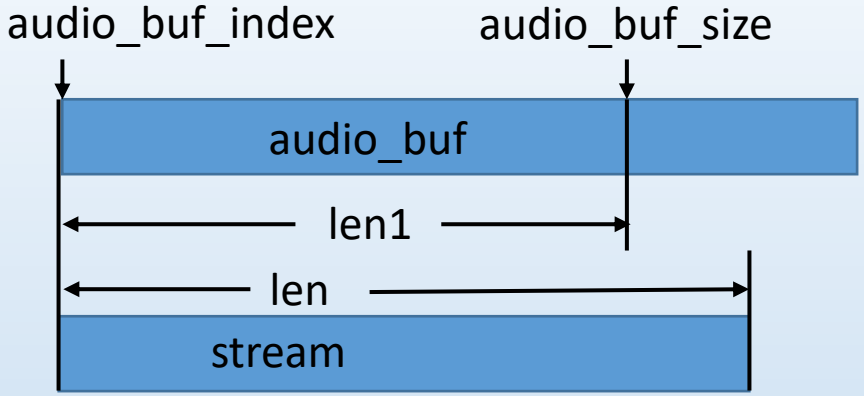

```
len1 = len;
memcpy(stream, (uint8_t *) audio_buf + audio_buf_index, len1);
len -= len1; stream += len1; audio_buf_index += len1;
```

下一次回调





len1 < len 时 表示此时完成stream部分复制 , 需要马上解码, 为剩余len-len1的长度复制

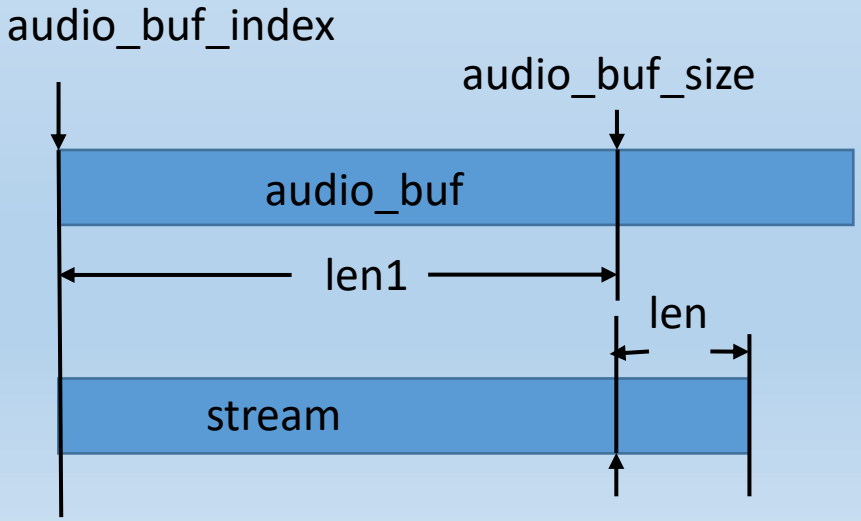


memcpy(stream, (uint8_t *) audio_buf + audio_buf_index, len1);
len -= len1; stream += len1; audio_buf_index += len1;

没有完成的需要下一次复制, 所以条件循环是 while(len)



复制完 audio_buf_index += len1;
之后, audio_buf_index >= audio_buf_size
触发解码, 数据拷贝 audio_buf 从0 开始, audio_buf_size结束
此时, audio_buf_index 归0



在经过上面的操作后, 还剩下 len = len - len1 需要复制

然后 len1 > len 取 len1 = len 复制
从而完成了此次stream复制, 缓冲区剩下的数据下一回调使用
流程如上

解码函数 int audio_decode_frame(AVCodecContext *pcodec_ctx, uint8_t *audio_buf, int buf_size)

1. 从队列中取出数据
2. 解码
3. swr_convert转换 并给audio_buf 赋值
4. 返回解码后用于音频长度